Nitte Meenakshi Institute of Technology

Yelahanka, Bangalore- 560064, Karnataka- India.

An Autonomous Institution with A$^+$ Grade by NAAC UGC, Approved by VTU, UGC, AICTE, Govt. of India.

Department of Electronics and Communication Engineering

AY: 2025-2026 (EVEN Semester)

Report submitted on the fulfillment of INTERNSHIP

DATA ENGINEERING

WEEKLY REPORT

Submitted By

Bhumika K.R **1NT22EC036**

Under the Guidance of

Mr. Amruth Ashok Gadag
Associate Professor

Mrs. Prajna K B
Assistant Professor

Dr.Anupama Sindgi
Assistant Professor

Department of Electronics and Communication Engineering
Nitte Meenakshi Institute of Technology,
Yelahanka, Bangalore-560064, Karnataka, India.

Contents

Sl. No.	Topics	Page No.
1	Introduction to Python Programming	1
2	Library Functions	2
2.1	NumPy and Matplotlib	2
2.2	Seaborn	9
2.3	IPyWidgets	10
2.4	Pandas	11
2.5	Streamlit	14
3	Introduction to Power BI	15
4	Building Enterprise Analytics Dashboard with Real-Time Data Streaming	16
4.1	Project Overview	16
4.2	Importing Dataset	16
4.3	Dataset Columns and Attributes	17
4.4	Importing Excel File into Power BI	18
4.5	Data Modeling and Relationships	18
4.6	Creating a New Table	19
4.7	Visualization and Reports	20
4.8	Publishing the Report to Power BI Server	23
4.9	Implementation Steps	24
5	Dashboard in a Day – Microsoft Learn	25
6	Basics of SQL	28
7	Conclusion	31

Chapter 1

INTRODUCTION TO PYTHON PROGRAMMING

As part of my data engineering internship, my initial task focused on gaining a solid foundation in Python programming and computing environments through hands-on practice with both Linux and Windows terminals. This phase aimed to strengthen my understanding of how Python operates at a fundamental level without relying on external libraries or packages, thereby deepening my knowledge of the language's core capabilities.

Python is an interpreted, high-level programming language well known for its simplicity, readability, and versatility. It is widely used in data engineering for data manipulation, automation, and workflow orchestration. In this introductory module, I worked directly in terminal environments to execute Python scripts, which helped me learn how the interpreter processes code and how command-line interfaces can simplify development and testing workflows.

Using the Linux terminal, I explored commands such as ls, cd to manage files and directories effectively. Running Python scripts using python3 filename.py allowed me to understand the execution process and system interactions at a lower level. The Linux environment also encouraged the use of shell scripting principles, permissions handling, and redirection operators to manage input and output data. These skills are essential for data engineers who often work in production servers or cloud-based Linux systems.

On the Windows terminal (Command Prompt and PowerShell), I performed similar tasks such as navigating directories, creating files, and running scripts. Working across both operating systems helped me appreciate their command differences and interoperability. This cross platform experience is valuable because real-world data pipelines often run on hybrid environments combining Windows-based tools with Linux-based servers.Regarding programming practice, I was restricted from using any external Python packages like NumPy or Pandas. Instead, I relied solely on Python's built-in functions and standard library modules to solve problems. I implemented simple programs for arithmetic operations, string manipulation, conditional branching, and file I/O operations. This approach helped reinforce my grasp of control structures, loops, data types, and error handling in pure Python.

CHAPTER 2

LIBRARY FUNCTIONS

2.1 NUMPY AND MATPLOTLIB

NumPy forms the foundation for numerical computing in Python. It supports high-performance array operations and mathematical computations. I practiced creating arrays using numpy.array() and performing vectorized operations, which are faster than conventional Python loops. I also used NumPy for calculating statistical measures such as mean, median, and standard deviation—operations essential for preprocessing data before analysis or visualization.

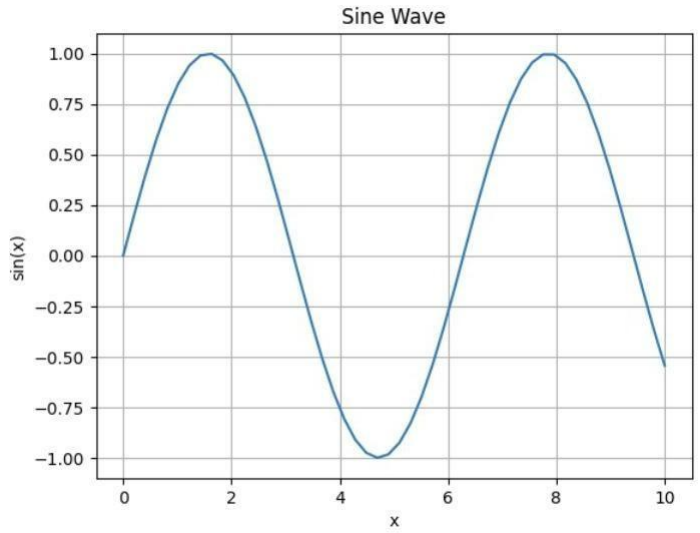
Matplotlib is a fundamental plotting library that allows users to create static, interactive, and publication-quality graphs. Using functions like plot(), bar() I visualized data distributions and trends. Configuring titles, legends, and axes labels helped me make data more interpretable, which is important for data-driven decision-making.

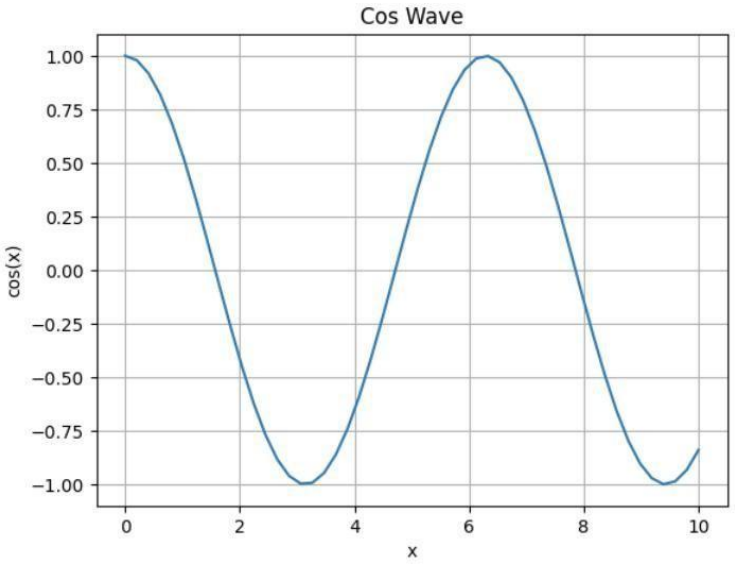
#PLOTTING SINE WAVE AND COS WAVE

```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(0,10,50)
# Sine wave
y_sin= np.sin(x)
plt.figure()
plt.plot(x, y_sin)
plt.title("Sine Wave")
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.grid()
plt.show()
#Cos wave
y_cos= np.cos(x)
plt.figure()
plt.plot(x, y_cos)
plt.title("Cos Wave")
plt.xlabel("x")
plt.ylabel("cos(x)")
plt.grid()
plt.show()
```

...





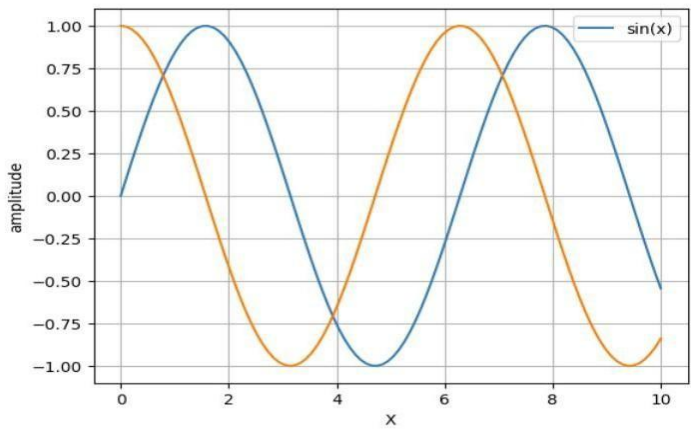
#PRINTING ARRAY

```
import numpy as np
arr_1=np.array([10,20,30,40,50])
arr_2=np.array([5,10,15,20,25])
print(f"Average of array1: {np.mean(arr_1)}")
print(f"Average of array2: {np.mean(arr_2)}")
print(f"Maximum of array1: {np.max(arr_1)}")
print(f"Maximum of array2: {np.max(arr_2)}")
```

```
Average of array1: 30.0
Average of array2: 15.0
Maximum of array1: 50
Maximum of array2: 25
```

#PLOTTING IN A SINGLE GRAPH

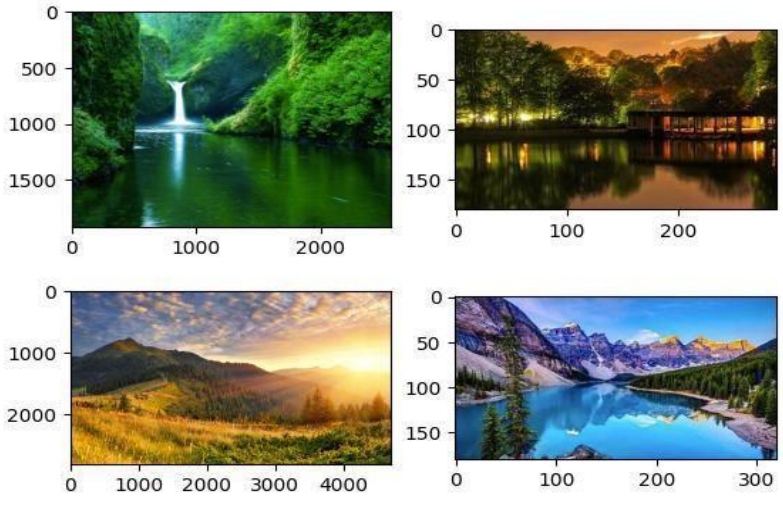
```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(0,10,100)
x1=np.sin(x)
y1=np.cos(x)
plt.plot(x,x1,label='sin(x)')
plt.plot(x,y1,label='cos(x)')
plt.xlabel('X')
plt.ylabel('amplitude')
plt.grid()
plt.legend(['sin(x)'])
plt.show()
```



#PLOTTING IMAGES

```
import matplotlib.pyplot as plt
import matplotlib.image as img
plt.subplot(2,2,1)
test=img.imread("/content/163868.jpg")
plt.imshow(test)
plt.subplot(2,2,2)
y=img.imread("/content/OIP (1).jpg")
plt.imshow(y)
plt.subplot(2,2,3)
z=img.imread("/content/mountain-scenery-morning-sun-rays-4k-kf.jpg")
plt.imshow(z)
plt.subplot(2,2,4)
q=img.imread("/content/OIP.jpg")
plt.imshow(q)
```

```
<matplotlib.image.AxesImage at 0x7cfa5db339b0>
```



#PLOTTING USING SUBPLOT

```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(0,2*np.pi,50)
plt.figure()

plt.subplot(2,3,1)
x1=np.exp(x)
plt.plot(x,x1, label='exp(x)')
plt.xlabel("x")
plt.ylabel("exp(x)")
plt.title("exponential")
plt.legend()

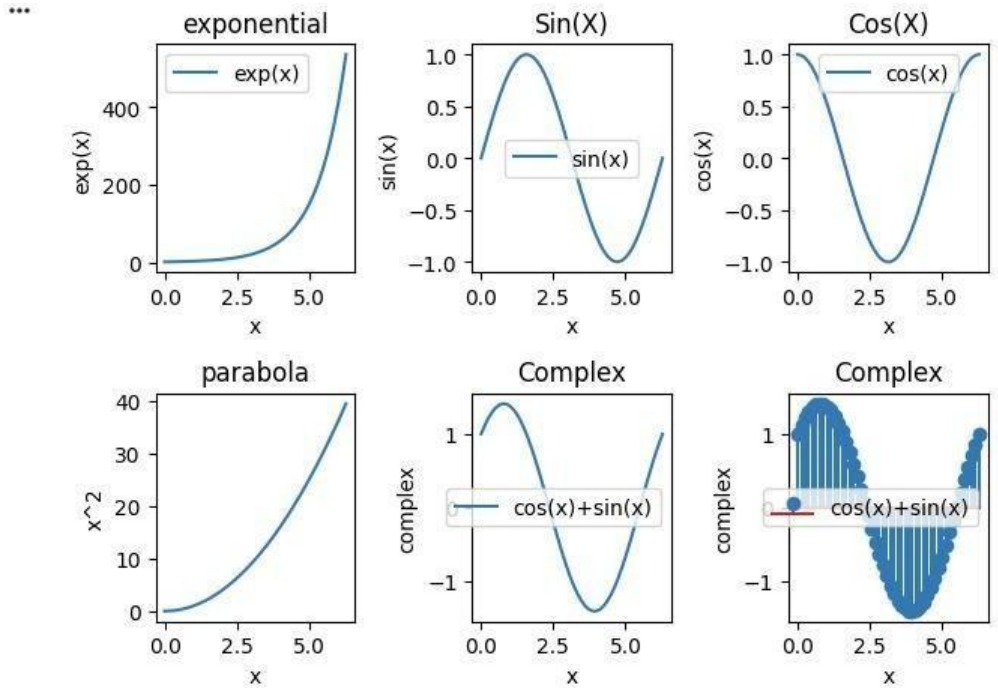
y1=np.sin(x)
plt.subplot(2,3,2)
plt.plot(x,y1, label='sin(x)')
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("Sin(X)")
plt.legend()

y2=np.cos(x)
plt.subplot(2,3,3)
plt.plot(x,y2, label='cos(x)')
plt.xlabel("x")
plt.ylabel("cos(x)")
plt.title("Cos(X)")
plt.legend()

y3=x**2
plt.subplot(2,3,4)
plt.plot(x,y3, label='x^2')
plt.xlabel("x")
plt.ylabel("x^2")
plt.title("parabola")
plt.legend
```

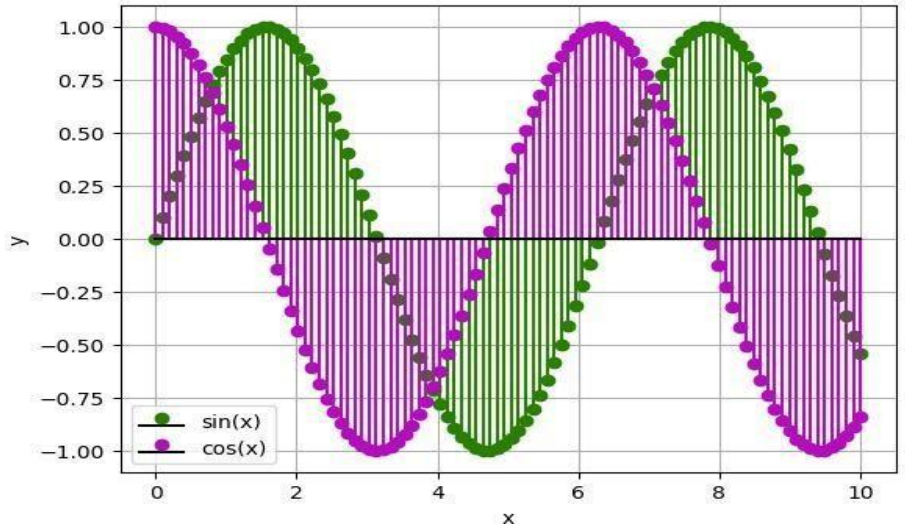
```
y4=y1+y2
plt.subplot(2,3,5)
plt.plot(x,y4, label='cos(x)+sin(x)')
plt.xlabel("x")
plt.ylabel("complex")
plt.title("Complex")
plt.legend()

y4=y1+y2
plt.subplot(2,3,6)
plt.stem(x,y4, label='cos(x)+sin(x)')
plt.xlabel("x")
plt.ylabel("complex")
plt.title("Complex")
plt.legend()
plt.tight_layout()
plt.show()
```

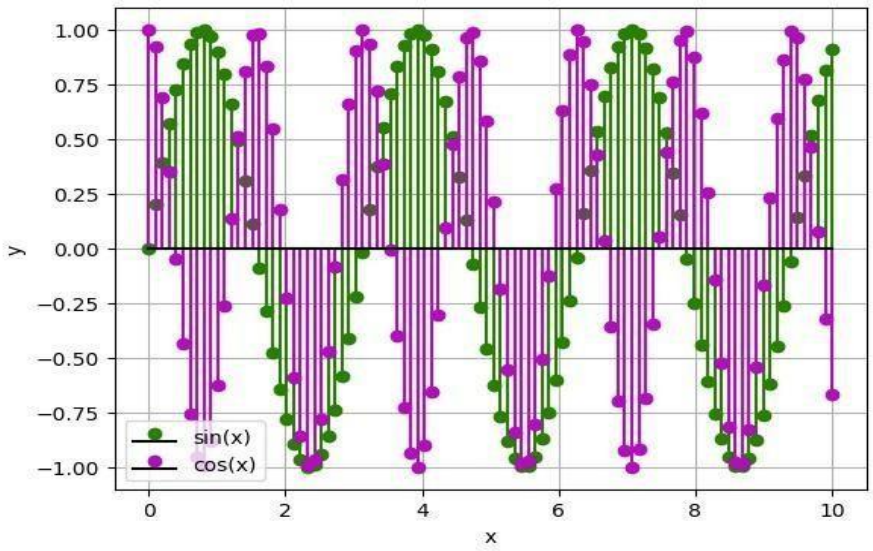


#PLOTTING USING STEM

```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(0,10,100)
x1=np.sin(x)
y1=np.cos(x)
plt.stem(x,x1,linefmt='g-', markerfmt='go', basefmt='k-', label='sin(x)')
plt.stem(x,y1,linefmt='m-', markerfmt='mo', basefmt='k-', label='cos(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.grid()
plt.legend()
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt
x=np.linspace(0,10,100)
x1=np.sin(2*x)
y1=np.cos(4*x)
plt.stem(x,x1,linefmt='g-', markerfmt='go', basefmt='k-', label='sin(x)')
plt.stem(x,y1,linefmt='m-', markerfmt='mo', basefmt='k-', label='cos(x)')
plt.xlabel('x')
plt.ylabel('y')
plt.grid()
plt.legend()
plt.show()
```



#PRINTING PRIME NUMBERS WITHIN 1-100

```
def is_prime(num):
  if num <= 1:
    return False
  for i in range(2, num):
    if num % i == 0:
      return False
  return True
for num in range(1, 101):
  if is_prime(num):
    print(num)
```

```
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
53
59
61
67
71
73
79
83
89
97
```

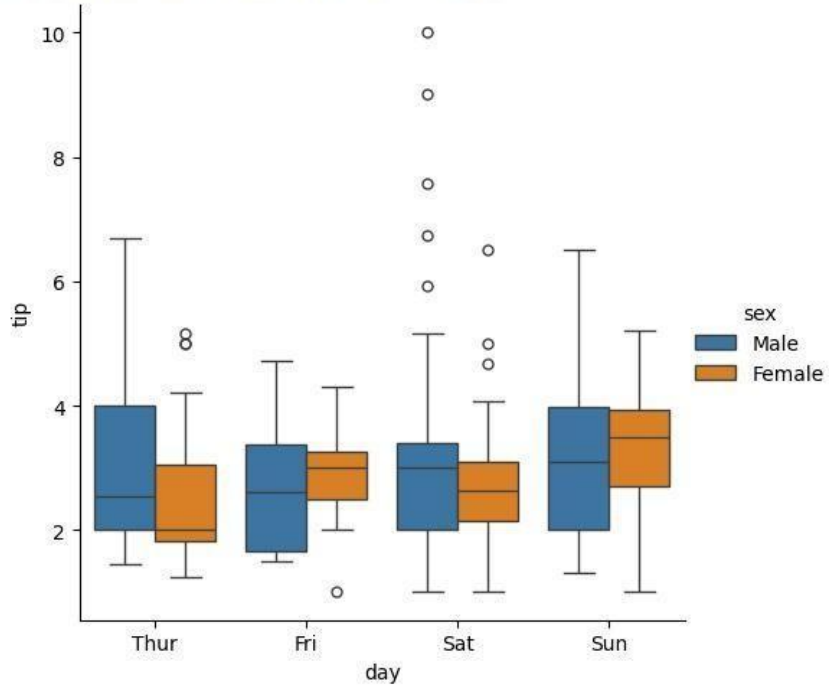
2.2 SEABORN

Built on top of Matplotlib, Seaborn simplifies statistical data visualization with aesthetic and informative graphics. I used it for visualizing relationships between variables through heatmaps, pair plots, and boxplots. Seaborn automatically integrates well with Pandas DataFrames, making
it ideal for exploratory data analysis.

We use in built dataset called as tips.

```
import seaborn as sns
%matplotlib inline
tips=sns.load_dataset('tips')
sns.catplot(x='day',y='tip',kind='box',data=tips,hue='sex')
```

```
<seaborn.axisgrid.FacetGrid at 0x7fa988d64890>
```



2.3 IPYWIDGETS

Widgets enable the creation of interactive controls directly within Colab notebooks. By using sliders, dropdowns, and buttons, I built interactive elements to modify parameters dynamically and visualize outcomes instantly. This enhanced usability and experimentation during analysis.

PLAYING ROCK,PAPER,SCISSOR

```
import ipywidgets as widgets
from IPython.display import display
import random
btns=widgets.ToggleButtons(
    options=['rock','paper','scissor'],
    description='Choose'
)
out=widgets.Label()
def play(change):
  comp=random.choice(['rock','paper','scissor'])
  user=btns.value
  if user==comp:
    out.value=f"Tie! Both chose {comp}"
  elif (user=='rock' and comp=='scissor') or \
       (user=='paper' and comp=='rock') or \
       (user=='scissor' and comp=='paper'):
       out.value=f"You win! Computer Plays {comp}"
  else:
    out.value=f"You lose! Computer Plays {comp}"
btns.observe(play,names='value')
display(btns,out)
```

```
Choose  rock  paper  scissor
You win! Computer Plays rock
```

#GUESS YOUR LUCK GAME

```
import ipywidgets as widgets
from IPython.display import display
import random

btn = widgets.Button(description="Guess Number (1-9)", button_style='info')
out = widgets.Label(value="Try your luck!")

def check_guess(b):
    secret = random.randint(1, 9)

    out.value = f"Secret was {secret}. You clicked the button!"

btn.on_click(check_guess)
display(btn, out)
```



The Secret was 5! Click the FAB to try again.

2.4 PANDAS

Pandas is a powerful open-source library designed for data manipulation and analysis. It introduces data structures like Series and DataFrame, which make it easier to organize and process tabular data. I learned to use functions such as read_csv(), head(), and describe() to load, explore, and summarize datasets. Pandas also provides built-in tools for cleaning data handling missing values, merging datasets, and reshaping tables efficiently.

#IMPORT ATTENDANCE AND MARKS CSV FILE

```
import pandas as pd
p=pd.read_csv("/attend.csv")
print(p)
print(p.shape)
print(p.size)
```

```
...    Unnamed: 0  Unnamed: 1          Unnamed: 2                  Unnamed: 3  \
0      Sl. No.         USN         Student Name  LINEAR INTEGRATED CIRCUITS
1         NaN         NaN                  NaN                         NaN
2         NaN         NaN                  NaN                         NaN
3           1  1NT23EC064    HARSHAVARDHAN B M                          84
4           2  1NT23EC065    HINDUJA GOWDA R N                          79
..        ...         ...                  ...                         ...
68         66  1NT24EC418  SATVIK SATEESH NAIK                          89
69         67  1NT24EC415           PUNITH R S                          79
70         68  1NT24EC416       SAI PRASAD R G                          84
71         69  1NT24EC404              DISHA N                          79
72         70  1NT22EC071             JEEVAN S                          68

                          Unnamed: 4  \
0   DISCRETE TIME SIGNAL PROCESSING
1                               NaN
2                               NaN
3                                85
4                                77
..                              ...
68                               95
69                               95
70                               82
71                               77
72                               82
```

```
                                    Unnamed: 5       Unnamed: 6  \
0   APPLICATIONS OF MACHINE LEARNING USING PYTHON  ELECTROMAGNETICS
1                                             NaN              NaN
2                                             NaN              NaN
3                                              88               73
4                                              94               82
..                                            ...              ...
68                                             87               91
69                                             91               77
70                                             75               77
71                                             72               91
72                                             84               77

                Unnamed: 7        Unnamed: 8  \
0   EMBEDDED SYSTEMS WITH C  DSD USING VERILOG
1                       NaN               NaN
2                       NaN               NaN
3                       NaN               NaN
4                        82               NaN
..                      ...               ...
68                      NaN               NaN
69                      NaN               NaN
70                      NaN               NaN
71                      NaN               NaN
72                      NaN               NaN

                       Unnamed: 9                          Unnamed: 10  \
0   INTRODUCTION TO DATA ANALYTICS  COMPUTER ORGANIZATION AND ARCHITECTURE
1                              NaN                                   NaN
2                              NaN                                   NaN
3                              NaN                                   NaN
4                              NaN                                   NaN
..                             ...                                   ...
68                             NaN                                    91
69                             NaN                                   NaN
70                              71                                   NaN
71                             NaN                                   NaN
72                             NaN                                    83

                                Unnamed: 11
0   FUNDAMENTALS OF 5G MOBILE NETWORKS
1                                  NaN
2                                  NaN
3                                   83
4                                  NaN
..                                 ...
68                                 NaN
69                                 NaN
70                                 NaN
71                                  83
72                                 NaN

[73 rows x 12 columns]
(73, 12)
876
```

```
import pandas as pd
p=pd.read_csv("/marks.csv")
p.head()
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10	Unnamed: 11
0	Sl. No.	USN	Student Name	LINEAR INTEGRATED CIRCUITS	DISCRETE TIME SIGNAL PROCESSING	APPLICATIONS OF MACHINE LEARNING USING PYTHON	ELECTROMAGNETICS	EMBEDDED SYSTEMS WITH C	DSD USING VERILOG	INTRODUCTION TO DATA ANALYTICS	COMPUTER ORGANIZATION AND ARCHITECTURE	FUNDAMENTALS OF 5G MOBILE NETWORKS
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	1	1NT23EC064	HARSHAVARDHAN B M	40	23	26	29	NaN	NaN	NaN	NaN	31
4	2	1NT23EC065	HINDUJA GOWDA R N	48	49	44	40	43	NaN	NaN	NaN	NaN

```
import pandas as pd
p=pd.read_csv("/marks.csv")
p.tail()
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10	Unnamed: 11
68	66	1NT24EC418	SATVIK SATEESH NAIK	5	17	9	1	NaN	NaN	NaN	23	NaN
69	67	1NT24EC415	PUNITH R S	22	23	11	4	17	NaN	NaN	NaN	NaN
70	68	1NT24EC416	SAI PRASAD R G	10	9	a	0	NaN	NaN	0	NaN	NaN
71	69	1NT24EC404	DISHA N	15	31	25	22	NaN	NaN	NaN	NaN	12
72	70	1NT22EC071	JEEVAN S	NE	NaN	a	15	NaN	NaN	NaN	22	NaN

```
import pandas as pd
p=pd.read_csv("/marks.csv")
p.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 73 entries, 0 to 72
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Unnamed: 0   71 non-null     object
 1   Unnamed: 1   71 non-null     object
 2   Unnamed: 2   71 non-null     object
 3   Unnamed: 3   71 non-null     object
 4   Unnamed: 4   65 non-null     object
 5   Unnamed: 5   71 non-null     object
 6   Unnamed: 6   63 non-null     object
 7   Unnamed: 7   25 non-null     object
 8   Unnamed: 8   11 non-null     object
 9   Unnamed: 9   15 non-null     object
 10  Unnamed: 10  10 non-null     object
 11  Unnamed: 11  12 non-null     object
dtypes: object(12)
memory usage: 7.0+ KB
```

```
import pandas as pd
p=pd.read_csv("/marks.csv")
p.describe()
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	Unnamed: 10	Unnamed: 11
count	71	71	71	71	65	71	63	25	11	15	10	12
unique	71	71	71	31	29	35	33	15	9	13	10	10
top	Sl. No.	USN	Student Name	NE	30	a	15	43	48	43	COMPUTER ORGANIZATION AND ARCHITECTURE	31
freq	1	1	1	6	5	10	6	5	2	2	1	3

```
import pandas as pd
p=pd.read_csv("/marks.csv")
print(p.iloc[:,4])
print(p.iloc[:,[2,4]])
```

```
0     DISCRETE TIME SIGNAL PROCESSING
1                                 NaN
2                                 NaN
3                                  23
4                                  49
                 ...
68                                 17
69                                 23
70                                  9
71                                 31
72                                NaN
Name: Unnamed: 4, Length: 73, dtype: object
             Unnamed: 2                       Unnamed: 4
0          Student Name  DISCRETE TIME SIGNAL PROCESSING
1                   NaN                              NaN
2                   NaN                              NaN
3     HARSHAVARDHAN B M                               23
4     HINDUJA GOWDA R N                               49
..                  ...                              ...
68  SATVIK SATEESH NAIK                               17
69           PUNITH R S                               23
70       SAI PRASAD R G                                9
71              DISHA N                               31
72            JEEVAN S                              NaN

[73 rows x 2 columns]
```

2.5 STREAMLIT

Finally, I explored Streamlit, a library for building interactive web applications using Python scripts. I developed simple dashboards that displayed visualizations and user inputs, transforming static analysis into interactive data applications.

```
pip install -q streamlit
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.0/9.0 MB 87.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 70.6 MB/s eta 0:00:00
```

```
%%writefile app1.py
import streamlit as st
from PIL import Image
st.title("What is your name?")
st.write("My name is Hasya")
```

```
Overwriting app1.py
```

```
!npm install -g localtunnel
!streamlit run app1.py --server.headless true &>/content/logs.txt &
!lt --port 8501 --print-requests
```

```
changed 22 packages in 702ms
3 packages are looking for funding
  run `npm fund` for details
your url is: https://new-plums-heal.loca.lt
^C
```

```
import requests
public_ip=requests.get("https://new-plums-heal.loca.lt").text
print("Your public IP is",public_ip)
```

CHAPTER 3

INTRODUCTION TO POWER BI

Power BI is a **business** analytics platform from Microsoft used to connect to data, transform it, and create interactive reports and dashboards for decision-making. It helps organizations turn raw data from multiple sources into visually rich insights that can be easily understood and shared across teams.

One of the key strengths of Power BI is its wide range of data connectivity options, allowing users to import or connect directly to databases, cloud services, Excel files, web data, and many other sources in a single environment. Using Power Query, data can be cleaned, merged, and transformed without writing complex code, which makes it suitable for both technical and non-technical users. The data model supports relationships between tables, calculated columns, and DAX measures, enabling powerful analytical calculations on top of large datasets.

Power BI also provides rich visualization capabilities, with many built-in charts, maps, tables, and custom visuals that can be arranged into interactive reports and dashboards. Users can apply filters, slicers, and drill-down actions to explore data in detail and uncover trends or anomalies quickly. These reports can then be published to the Power BI service, where they are accessible via browser or mobile apps, supporting secure sharing, role-based access, and collaboration within workspaces.

Another important feature is its support for scheduled and real-time data refresh, ensuring that dashboards always display up-to-date information for monitoring key performance indicators. Organizations can also leverage advanced capabilities such as AI-driven insights, integration with Excel and other Microsoft 365 tools, and embedding reports into applications and websites. Together, these features make Power BI a comprehensive platform for end-to-end business intelligence, from data ingestion to interactive visualization and enterprise-level sharing.

CHAPTER 4

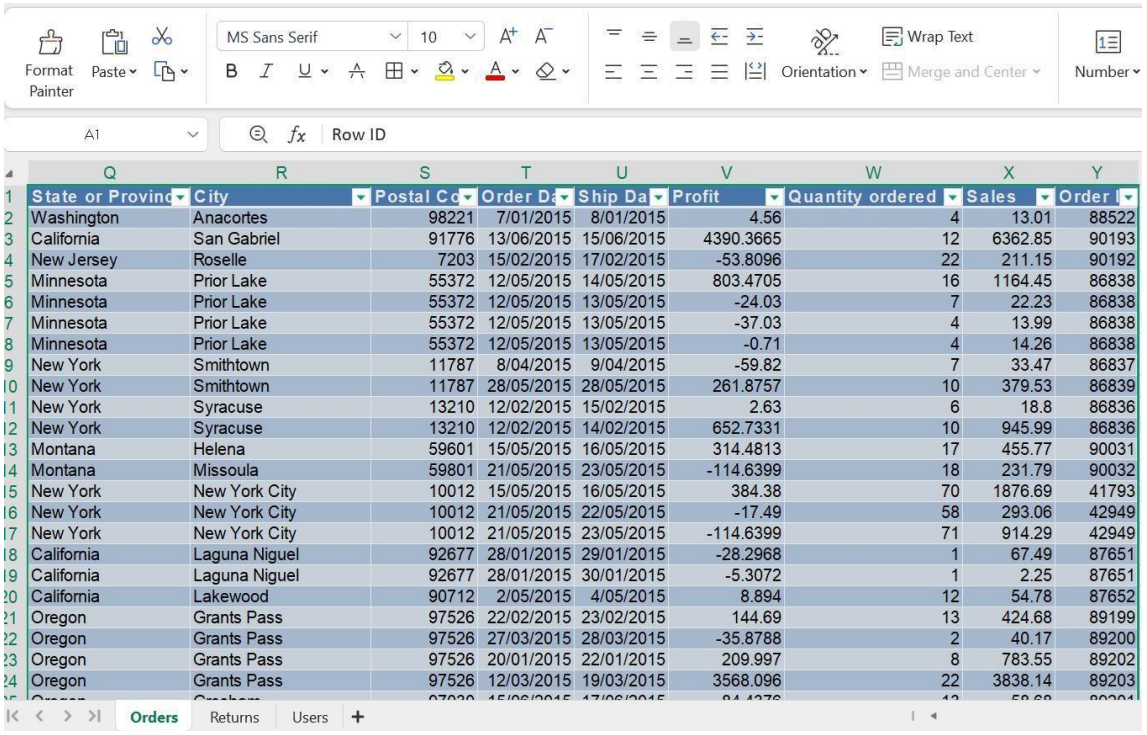
BUILDING ENTERPRISE ANALYTICS DASHBOARDS WITH REAL-TIME DATA STREAMING

4.1 PROJECT OVERVIEW

Develop a comprehensive business intelligence solution using Power BI that integrates multiple data sources, creates interactive dashboards, and implements real-time data streaming capabilities.

4.2 IMPORTING DATASET

The Excel file contained a single worksheet with transactional data spanning multiple years (2015 onwards).



	Q	R	S	T	U	V	W	X	Y
1	State or Provinc	City	Postal Co	Order Da	Ship Da	Profit	Quantity ordered	Sales	Order I
2	Washington	Anacortes	98221	7/01/2015	8/01/2015	4.56	4	13.01	88522
3	California	San Gabriel	91776	13/06/2015	15/06/2015	4390.3665	12	6362.85	90193
4	New Jersey	Roselle	7203	15/02/2015	17/02/2015	-53.8096	22	211.15	90192
5	Minnesota	Prior Lake	55372	12/05/2015	14/05/2015	803.4705	16	1164.45	86838
6	Minnesota	Prior Lake	55372	12/05/2015	13/05/2015	-24.03	7	22.23	86838
7	Minnesota	Prior Lake	55372	12/05/2015	13/05/2015	-37.03	4	13.99	86838
8	Minnesota	Prior Lake	55372	12/05/2015	13/05/2015	-0.71	4	14.26	86838
9	New York	Smithtown	11787	8/04/2015	9/04/2015	-59.82	7	33.47	86837
10	New York	Smithtown	11787	28/05/2015	28/05/2015	261.8757	10	379.53	86839
11	New York	Syracuse	13210	12/02/2015	15/02/2015	2.63	6	18.8	86836
12	New York	Syracuse	13210	12/02/2015	14/02/2015	652.7331	10	945.99	86836
13	Montana	Helena	59601	15/05/2015	16/05/2015	314.4813	17	455.77	90031
14	Montana	Missoula	59801	21/05/2015	23/05/2015	-114.6399	18	231.79	90032
15	New York	New York City	10012	15/05/2015	16/05/2015	384.38	70	1876.69	41793
16	New York	New York City	10012	21/05/2015	22/05/2015	-17.49	58	293.06	42949
17	New York	New York City	10012	21/05/2015	23/05/2015	-114.6399	71	914.29	42949
18	California	Laguna Niguel	92677	28/01/2015	29/01/2015	-28.2968	1	67.49	87651
19	California	Laguna Niguel	92677	28/01/2015	30/01/2015	-5.3072	1	2.25	87651
20	California	Lakewood	90712	2/05/2015	4/05/2015	8.894	12	54.78	87652
21	Oregon	Grants Pass	97526	22/02/2015	23/02/2015	144.69	13	424.68	89199
22	Oregon	Grants Pass	97526	27/03/2015	28/03/2015	-35.8788	2	40.17	89200
23	Oregon	Grants Pass	97526	20/01/2015	22/01/2015	209.997	8	783.55	89202
24	Oregon	Grants Pass	97526	12/03/2015	19/03/2015	3568.096	22	3838.14	89203

Order I	Status
65	Returned
612	Returned
614	Returned
678	Returned
710	Returned
740	Returned
775	Returned
833	Returned
902	Returned
3300	Returned
3456	Returned
3525	Returned
3589	Returned
3687	Returned
3777	Returned
3783	Returned
4006	Returned
4037	Returned
4230	Returned
4261	Returned
4391	Returned
4610	Returned

Region	Manag
Central	Chris
East	Erin
South	Sam
West	William

4.3 DATASET COLUMNS AND ATTRIBUTES

Column Name	Data Type	Description
Order ID	Text	Unique identifier for each order
Product Priority	Text	Priority level (Critical, High, Medium, Low, Not Specified)
Discount	Decimal	Discount percentage applied to the order
Unit Price	Currency	Price per unit of the product
Manufacturing Price	Currency	Cost to manufacture the product
Customer ID	Number	Unique identifier for each customer
Customer Name	Text	Full name of the customer
Shipping Mode	Text	Method of shipment (Regular Air, Express Air, Delivery Truck)
Segment	Text	Customer segment (Corporate, Small Business, Consumer, Home Office)
Category	Text	Product category (Office Supplies, Technology, Furniture)
Sub-Category	Text	Detailed product classification (e.g., Binders, Telephones)
Container	Text	Packaging size (Small Box, Large Box, Jumbo Box, Wrap Bag, etc.)
Product Name	Text	Full description of the product
Profit	Currency	Profit amount for the transaction
Country	Text	Country of operation (United States)
Region	Text	Geographic region (East, West, Central, South)
State	Text	State abbreviation or full name
City	Text	City/Location name
Postal Code	Number	ZIP/Postal code
Order Date	Date	Date when order was placed (Excel serial format: 42028, etc.)
Ship Date	Date	Date when order was shipped
Profit (numeric)	Currency	Profit value in numerical format
Quantity	Number	Number of units ordered
Sales	Currency	Total sales amount for the transaction

4.4 IMPORT EXCEL FILE INTO POWER BI

Step 1: Launch Power BI Desktop

4.4.1 Opened Power BI Desktop application

4.4.2 Started a new blank project

Step 2: Access Data Connector

4.4.3 Clicked on **"Get Data"** in the Home ribbon

4.4.4 Selected **"Excel"** from the data source options

Step 3: Browse and Select File

4.4.5 Navigated to the file location containing `S01-Welcome.SuperStoreUS1-2015.xlsx`

4.4.6 Selected the Excel workbook

4.4.7 Clicked **"Open"** to establish connection

Step 4: Preview Data

4.4.8 Power BI displayed the available tables/sheets in the Excel file

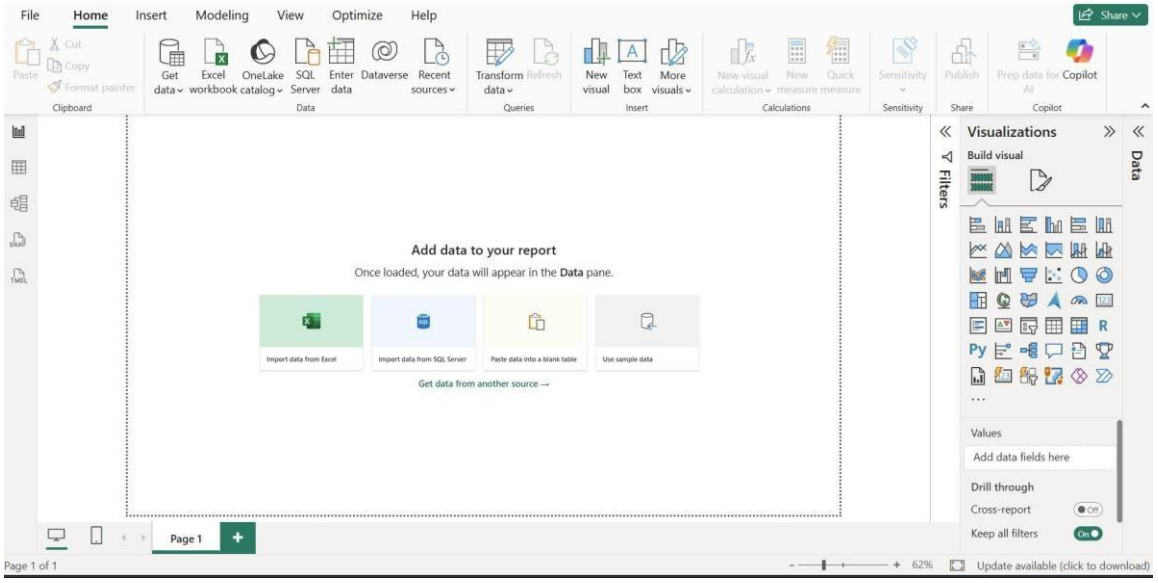
4.4.9 Selected the main data sheet containing sales transactions

4.4.10 Reviewed the data preview to confirm correct table selection

Step 5: Load Data

4.4.11 Clicked **"Load"** to import the entire dataset into Power BI

4.4.12 Alternatively, clicked **"Transform Data"** to access Power Query for initial transformations.

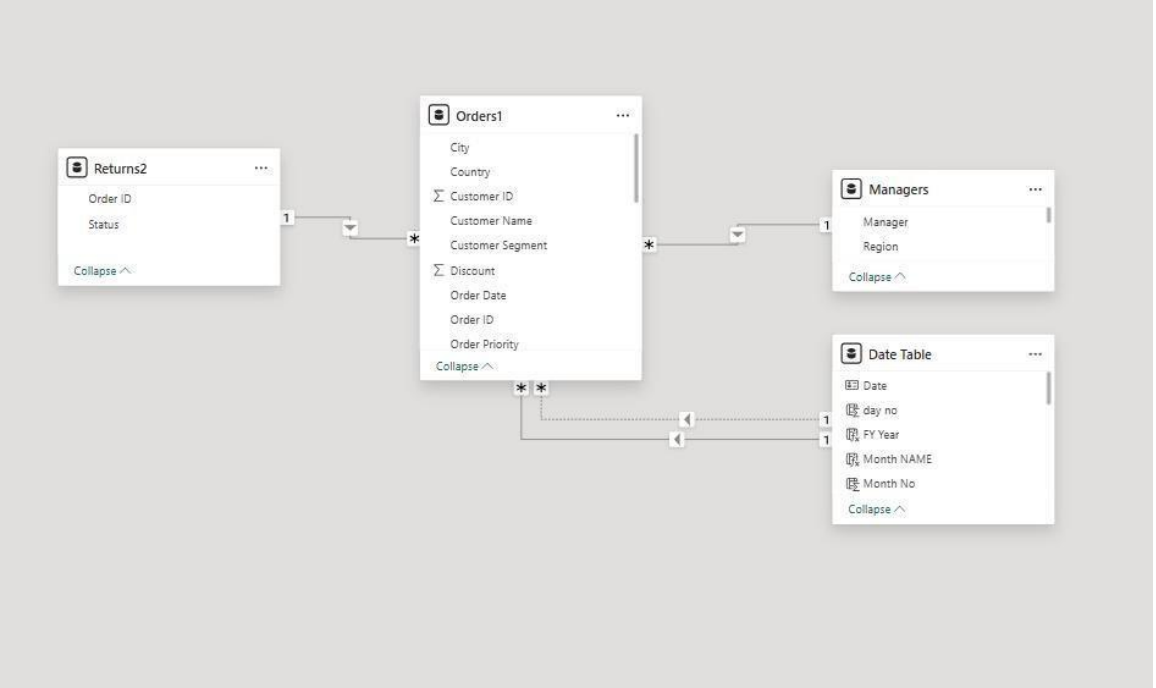


4.5 DATA MODELLING AND RELATIONSHIPS

Implemented a **Star Schema** design with:

4.5.1 **Fact Table:** Sales (transactional data)

4.5.2 **Dimension Tables:** Date, Customer, Product, Region



Relationship	From Table	To Table	Type	Cardinality
Product Lookup	Sales	Product	One-to-Many	1:*
Customer Lookup	Sales	Customer	One-to-Many	1:*
Date Lookup	Sales	Date	One-to-Many	1:*
Region Lookup	Customer	Region	One-to-Many	1:*

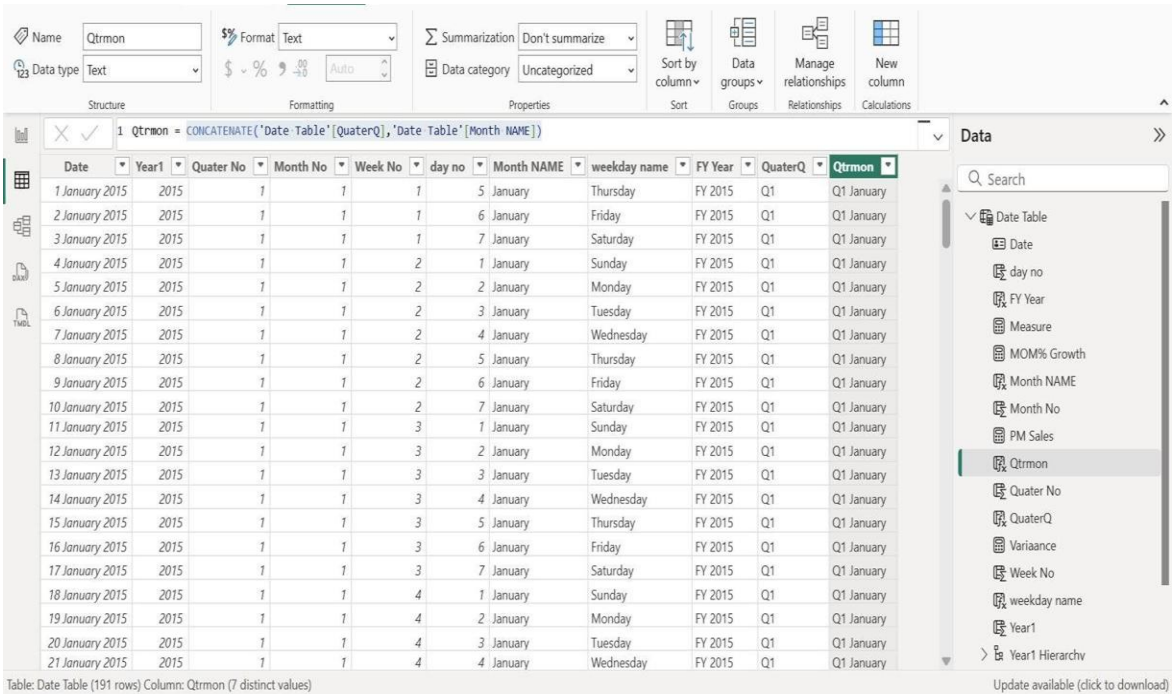
4.6 CREATE A NEW TABLE

A custom date table was created using DAX to enable time-based analysis:

```
1  Date Table = CALENDAR(DATE(2015,1,1),DATE(2015,7,10))
```

Column Name	DAX Formula
Year	YEAR('Date Table'[Date])
Quarter No	QUARTER('Date Table'[Date])
Month No	MONTH('Date Table'[Date])
Week No	WEEKNUM('Date Table'[Date])
Day No	WEEKDAY('Date Table'[Date])
Month Name	FORMAT('Date Table'[Date],"mmmm")

Weekday Name	FORMAT('Date Table'[Date],"DDDD")
FY Year	"FY" & " " & 'Date Table'[Year1]
Quarter Q	FORMAT('Date Table'[Date],"\Qq ")
Quarter Month	CONCATENATE('Date Table'[QuaterQ],'Date Table'[Month NAME])



Date	Year1	Quater No	Month No	Week No	day no	Month NAME	weekday name	FY Year	QuaterQ	Qtrmon
1 January 2015	2015	1	1	1	5	January	Thursday	FY 2015	Q1	Q1 January
2 January 2015	2015	1	1	1	6	January	Friday	FY 2015	Q1	Q1 January
3 January 2015	2015	1	1	1	7	January	Saturday	FY 2015	Q1	Q1 January
4 January 2015	2015	1	1	2	1	January	Sunday	FY 2015	Q1	Q1 January
5 January 2015	2015	1	1	2	2	January	Monday	FY 2015	Q1	Q1 January
6 January 2015	2015	1	1	2	3	January	Tuesday	FY 2015	Q1	Q1 January
7 January 2015	2015	1	1	2	4	January	Wednesday	FY 2015	Q1	Q1 January
8 January 2015	2015	1	1	2	5	January	Thursday	FY 2015	Q1	Q1 January
9 January 2015	2015	1	1	2	6	January	Friday	FY 2015	Q1	Q1 January
10 January 2015	2015	1	1	2	7	January	Saturday	FY 2015	Q1	Q1 January
11 January 2015	2015	1	1	3	1	January	Sunday	FY 2015	Q1	Q1 January
12 January 2015	2015	1	1	3	2	January	Monday	FY 2015	Q1	Q1 January
13 January 2015	2015	1	1	3	3	January	Tuesday	FY 2015	Q1	Q1 January
14 January 2015	2015	1	1	3	4	January	Wednesday	FY 2015	Q1	Q1 January
15 January 2015	2015	1	1	3	5	January	Thursday	FY 2015	Q1	Q1 January
16 January 2015	2015	1	1	3	6	January	Friday	FY 2015	Q1	Q1 January
17 January 2015	2015	1	1	3	7	January	Saturday	FY 2015	Q1	Q1 January
18 January 2015	2015	1	1	4	1	January	Sunday	FY 2015	Q1	Q1 January
19 January 2015	2015	1	1	4	2	January	Monday	FY 2015	Q1	Q1 January
20 January 2015	2015	1	1	4	3	January	Tuesday	FY 2015	Q1	Q1 January
21 January 2015	2015	1	1	4	4	January	Wednesday	FY 2015	Q1	Q1 January

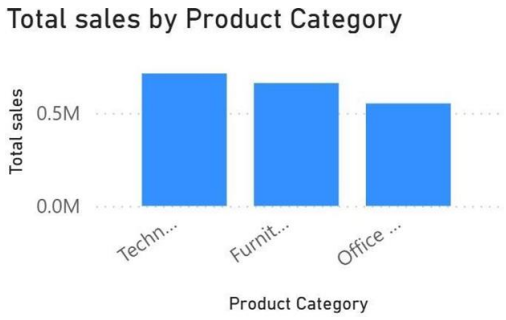
4.7 VISUALIZATIONS AND REPORTS

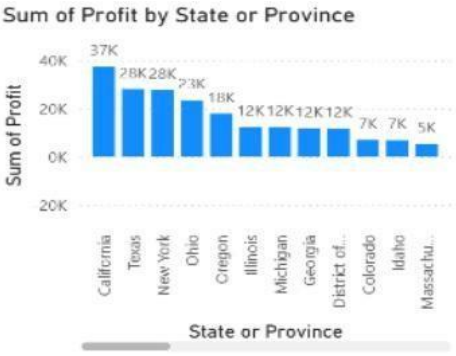
Selected appropriate visualization types based on data and insights required:

1. TABLE

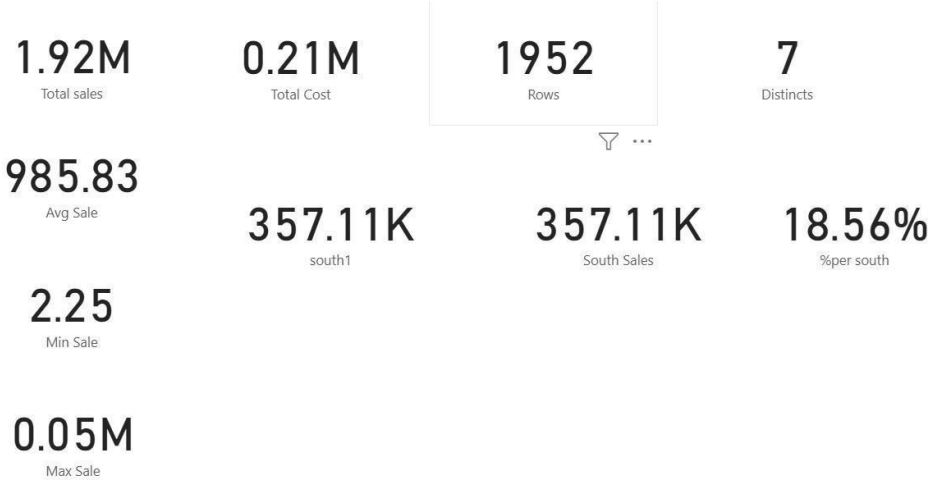
Manager	Sum of Sales
Chris	448,284.70
Erin	592,171.49
Sam	357,105.12
William	526,776.57
Total	**1,924,337.88**

2. STACKED COLUMN CHART





3. CARD



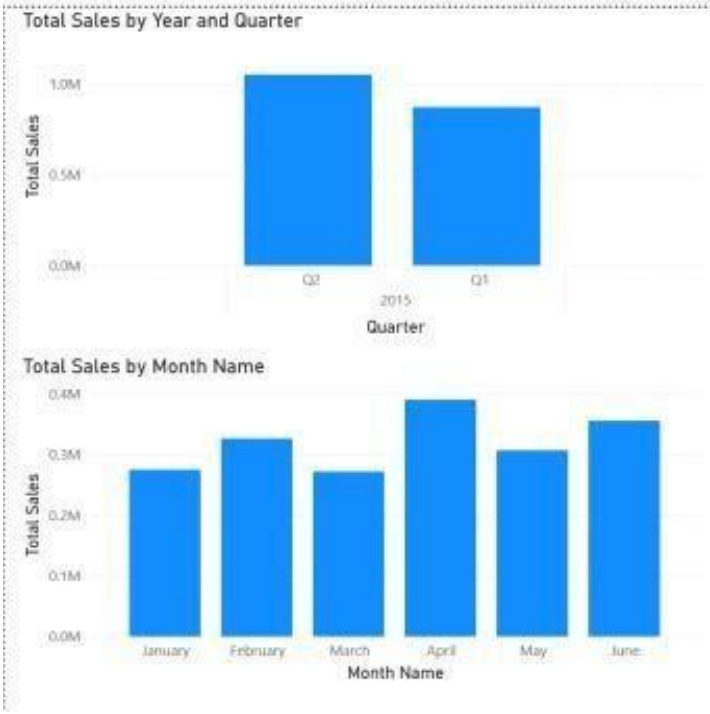
4. LINE AND CLUSTERED COLUMN CHART



5. TABLE WITH PROFIT RATIO

Month No	Total Sales	PM Sales	Variance	MOM% Growth
1	2,74,766.92		2,74,766.92	100.00%
2	3,26,101.47	2,74,766.92	51,334.55	15.74%
3	2,71,696.67	3,26,101.47	-54,404.80	-20.02%
4	3,89,831.95	2,71,696.67	1,18,135.28	30.30%
5	3,06,572.07	3,89,831.95	-83,259.88	-27.16%
6	3,55,368.80	3,06,572.07	48,796.73	13.73%
7		3,55,368.80	-3,55,368.80	0.00%
Total	**19,24,337.88**		**19,24,337.88**	**100.00%**

6. CLUSTERED COLUMN CHART



7. NARRATIVE

Total Total sales was higher for (632,274.88) than Returned (29,665.99).

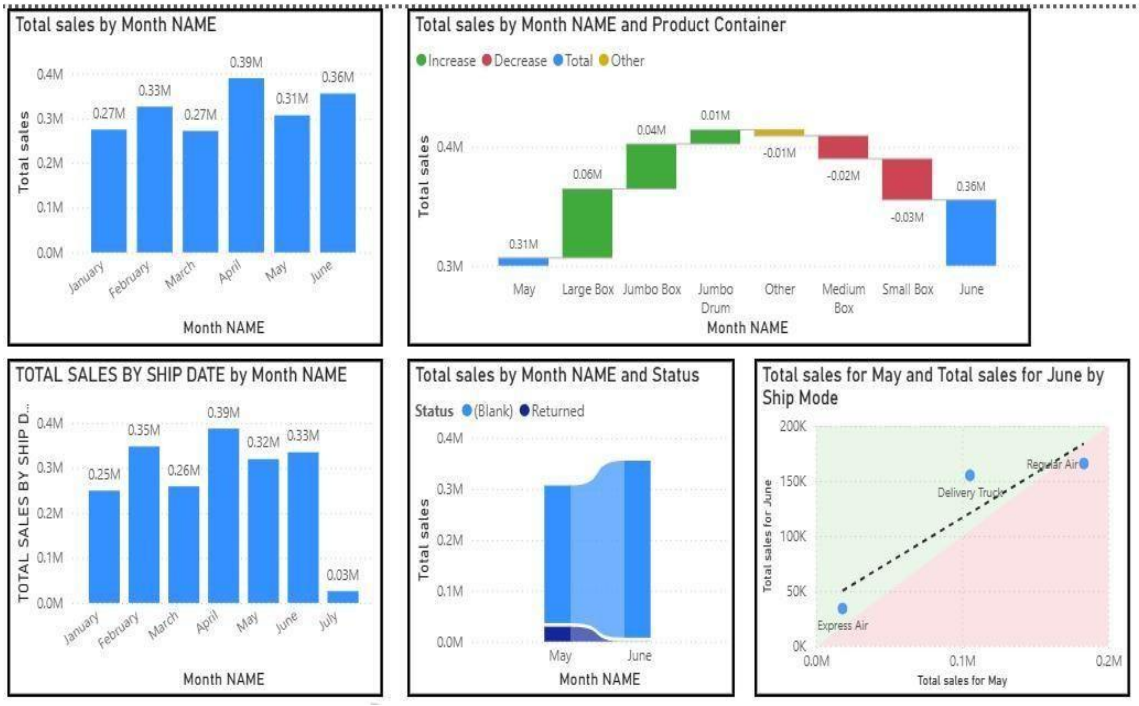
June in Status made up 53.62% of Total sales.

Average Total sales was higher for (316,137.44) than Returned (14,833.00).

Total sales for and Returned diverged the most when the Month NAME was June, when were 354,492.30 higher than Returned.

At 389,831.95, April had the highest Total sales and was 43.48% higher than March, which had the lowest Total sales at 271,696.67.

8. REPORT WITH PROPER BORDER ALIGNMET



4.8 PUBLISHING THE REPORT TO POWER BI SERVER

Step 1: Prepare for Publishing

4.8.1 Reviewed all reports and dashboards for accuracy

4.8.2 Verified data connections and credentials

4.8.3 Optimized performance of queries

Step 2: Configure Power BI Service Settings

4.8.4 Signed into Power BI Service (app.powerbi.com)

4.8.5 Created a new workspace: "DAY 3"

4.8.6 Set appropriate access permissions

Step 3: Publish Report

4.8.7 In Power BI Desktop: File → Publish

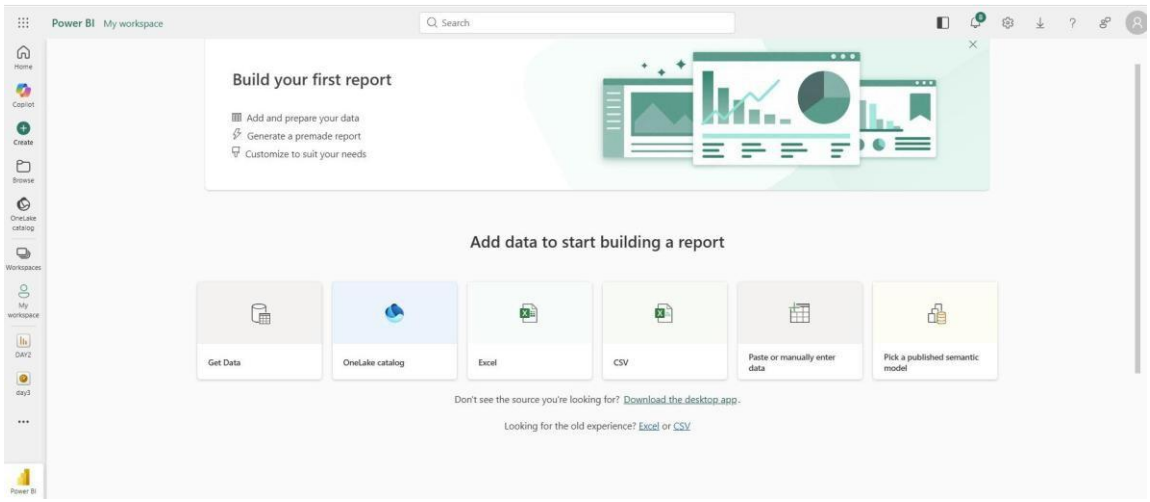
4.8.8 Selected target workspace: "DAY 3"

4.8.9 Confirmed publication of all report pages and datasets

Step 4: Configure Dataset Settings

4.8.10 Configured data source credentials

4.8.11 Enabled gateway connectivity for on-premises data.



4.9 Implementation Steps

Step 1: Create PubNub Account

4.9.1 Registered at pubnub.com

4.9.2 Created a new app and configured keys

4.9.3 Obtained Publish and Subscribe keys

Step 2: Set Up Streaming Dataset in Power BI

4.9.4 In Power BI Service: Create → Streaming dataset

4.9.5 Selected "API" as data source

4.9.6 Defined schema with columns



CHAPTER 5

DASHBOARD IN A DAY BY MICROSOFT LEARN

1. Program Overview: One-day intensive Microsoft Learn training to create interactive business dashboards using Power BI Desktop, designed for beginners to intermediate users needing practical BI skills.

2.Dashboard Fundamentals: Learned that dashboards provide quick at-a-glance summaries with key metrics, unlike detailed reports; focused on visual hierarchy, color theory, and design principles to communicate insights effectively.

3. Power BI Interface: Mastered Power BI Desktop interface with tabs (Home, Insert, Data, Modeling, View) and working panes (Data, Visualizations, Filters) across three views: Report, Data, and Model.

4. Data Connectivity: Connected to multiple data sources (Excel, SQL Server, CSV, web) using import/live query modes; configured refresh schedules and managed credentials securely for real-time data access.

5. Data Transformation: Used Power Query to clean, transform, and prepare data by removing duplicates, splitting/merging columns, filtering rows, and creating helper columns; optimized queries for performance.

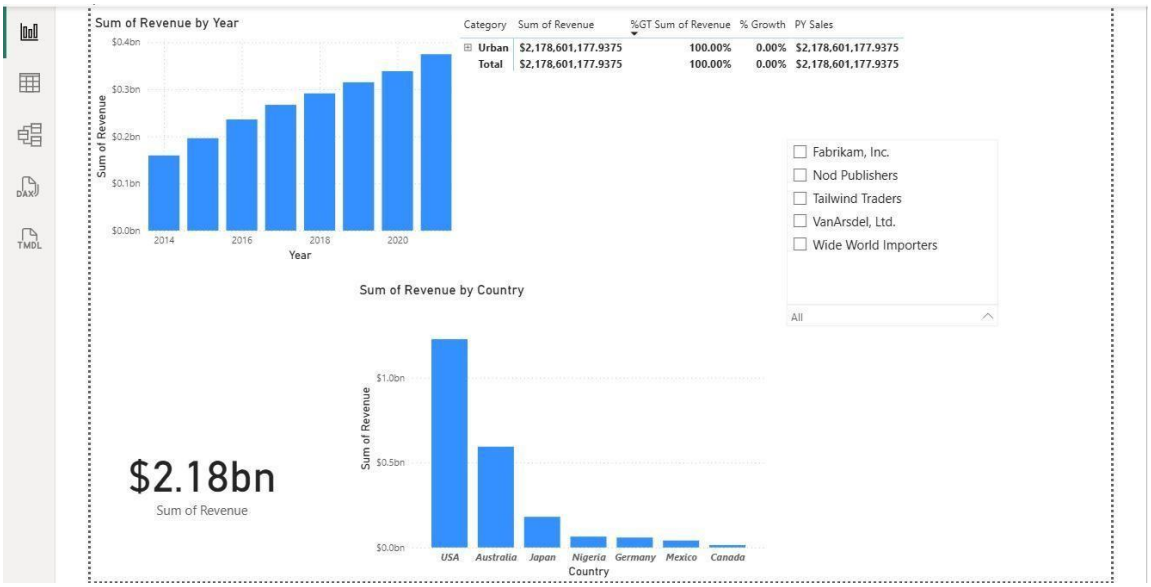
6. Data Modeling: Built star schema with fact table (Sales) and dimension tables (Customers, Products, Dates, Regions); established one-to-many relationships and created hierarchies for drill-down capabilities.

7. Visualizations & Charts: Created 10+ chart types (bar, line, pie, scatter, map, KPI, gauge, waterfall) with corporate branding colors, data labels, and customized formatting for clear data storytelling.

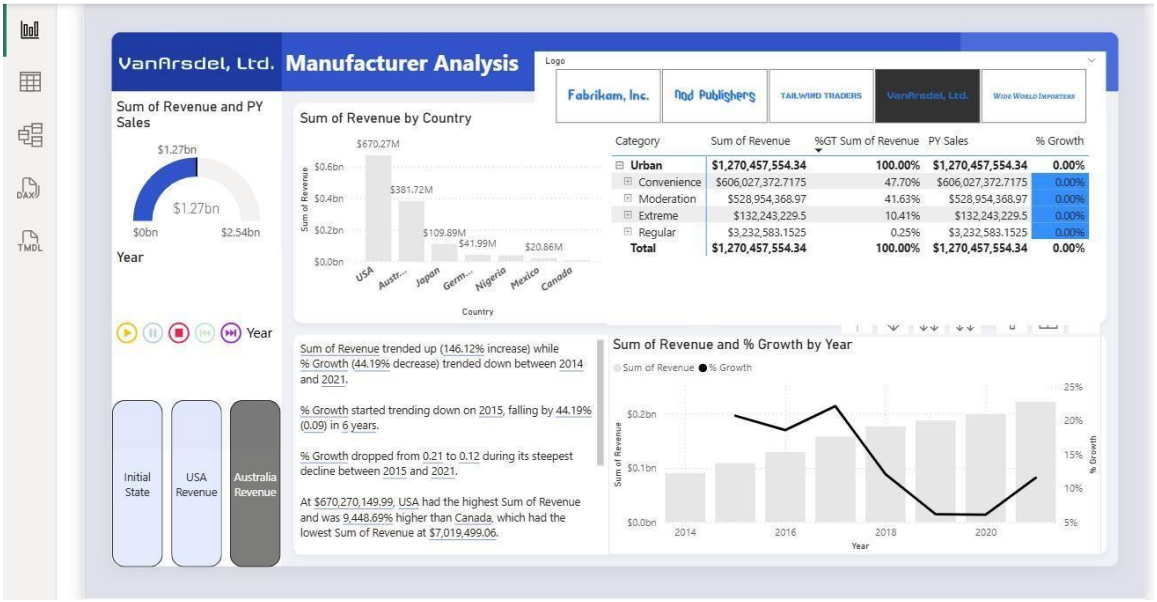
8. DAX Calculations: Wrote advanced formulas using aggregation (SUM, AVERAGE), time intelligence (YTD, PREVIOUSMONTH), conditional logic (IF, SWITCH), and filtering (CALCULATE, RANKX) for complex business metrics.

9. Interactive Dashboard: Implemented slicers for date/region filtering, cross-filtering between visuals, drill-down hierarchies, bookmarks for navigation, and toggle buttons; completed a full Sales Performance Dashboard in 8 hours.

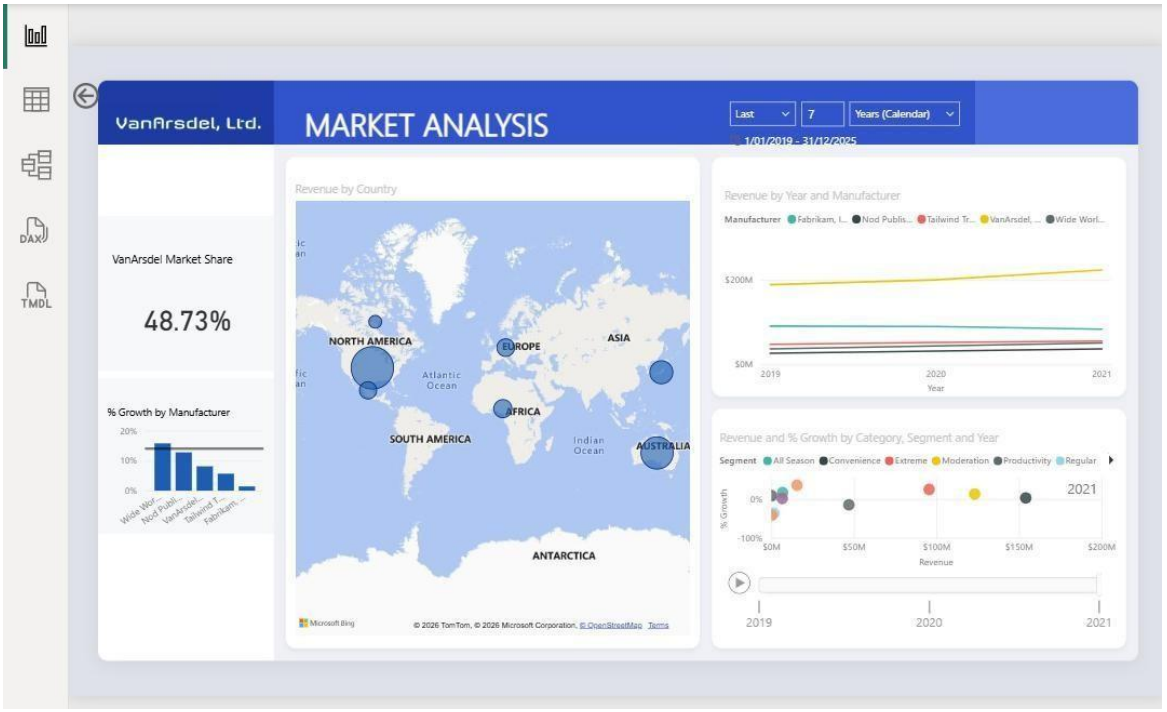
5.1 DIFFERENT VISUALIZATION



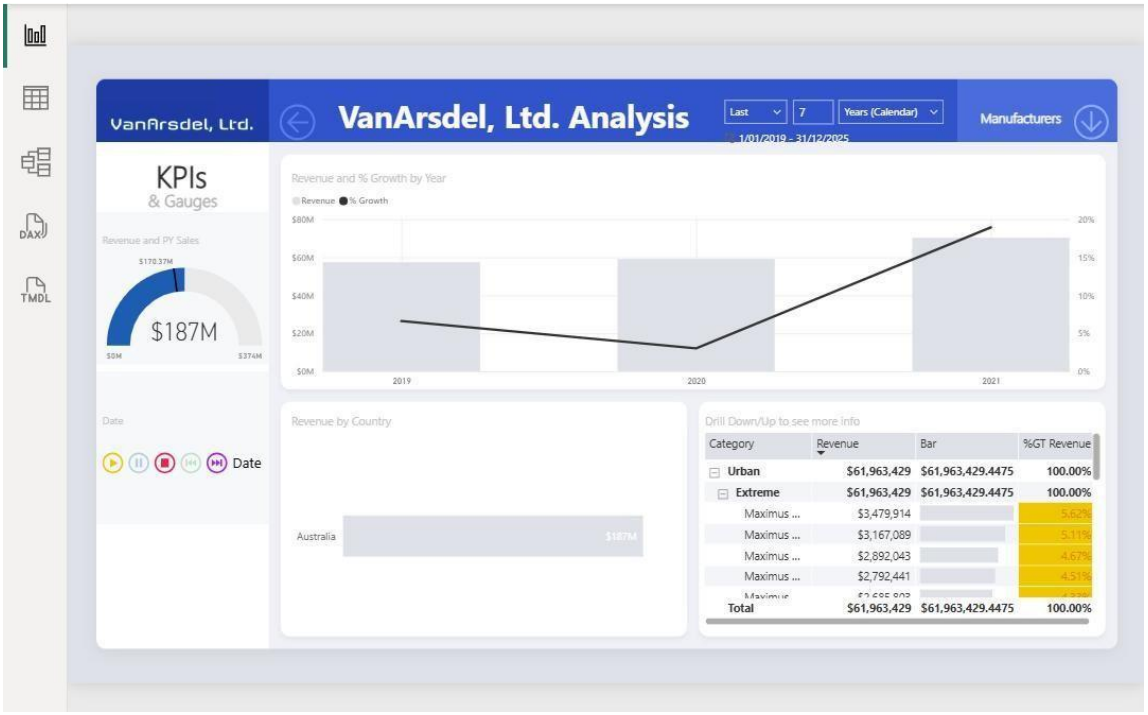
5.2 ANALYSIS BY MANUFACTURER



5.3 MARKET ANALYSIS



5.4 MANUFACTURER ANALYSIS



Chapter 6

BASICS OF SQL

SQL, or Structured Query Language, is a standardized programming language for interacting with relational databases.
It handles data organization, storage, and retrieval in tables with rows and column

SELECT

- Specifies which columns or expressions appear in the result set.
- Can return all columns or only selected columns.
- Supports calculated expressions and built-in functions.
- Can remove duplicates and rename columns using options like DISTINCT and aliases.

FROM

- Identifies the table or tables that provide the data.
- Allows combining multiple tables using joins.
- Accepts views, subqueries, and other row-producing sources.
- Always works together with a SELECT clause in a query. WHERE
- Filters rows coming from the FROM source.
- Keeps only rows where the condition evaluates to true.
- Uses comparison, logical, pattern, and set operators.
- Is optional; if omitted, all rows from the source are returned.

6.1 Introduction to Relational Integrity

The primary goal of a relational database is to maintain "Single Source of Truth." This is achieved through constraints that prevent human error and logical inconsistencies.

6.1.1 Primary Key (PK) & Entity Integrity

A Primary Key ensures that every record is unique and identifiable. During the experiment, we observed:

- **Uniqueness:** Attempting to insert a duplicate unique identifier (USN) resulted in a Duplicate Entry error.
- **Non-Nullability:** The system rejected records without a PK, ensuring no "ghost" records exist.

6.1.2 Foreign Key (FK) & Referential Integrity

Foreign keys create the "glue" between tables. We tested two primary behaviors:

1. **Restrict (Default):** The database blocked the deletion of a student if they had existing marks, preventing "orphaned" data.
2. **Cascade:** In later stages, we configured the system to automatically delete child records when a parent was removed, maintaining a clean database state.

6.2. Advanced Data Manipulation & Retrieval

Efficient data management requires more than simple storage; it requires the ability to reconstruct complex relationships via Joins and Aggregate functions.

6.2.1 The Logic of Joins

We implemented several Join types to combine data from the Student, Department, and Course tables:

- **Inner Join:** Matched only the students who belong to a department.
- **Left Join:** Ensured that even students not yet assigned to a department appeared in the report.
- **Self Join:** Used to compare rows within the same table, such as finding students in the same age group.

6.2.2 Aggregation and Analytics

By using functions like MAX(), MIN(), and AVG(), we transformed raw data into insights, such as identifying the highest marks in a course or the average age of a department’s student body.

6.3. Data Abstraction and Automation

To enhance user experience and maintain logs, we utilized **Views** and **Triggers**.

6.3.1 Virtual Tables (Views)

A View (StudentInfo) was created to hide the complexity of multiple Joins. Users can query the

View as a single table, which enhances security by hiding sensitive columns (like internal IDs) while providing necessary information.

6.3.2 Automated Auditing (Triggers)

To track changes in student performance, we implemented an AFTER UPDATE trigger.

- **Mechanism:** Whenever a grade is modified, the trigger automatically pushes the Old_Value, New_Value, and Timestamp into a Marks_Audit table.
- **Benefit:** This provides a non-repudiable log of all changes, crucial for academic and financial integrity.

6.4. Database Security & User Management

Security in DBMS is managed through **Role-Based Access Control (RBAC)**. This prevents unauthorized data modification while allowing users to perform their specific duties.

6.4.1 Implementation of Roles

We defined two distinct roles to simulate a university hierarchy:

1. **Admin Role:** Granted ALL PRIVILEGES to manage schema changes and user accounts.
2. **DataEntry Role:** Restricted to INSERT and SELECT. This user can add new students but cannot delete existing records or drop tables.

6.4.2 Security Verification Table

Operation	Admin User	Data Entry User	Security Logic
Drop Table	Permitted	**Denied**	Only Admins should alter structure.
Delete Row	Permitted	**Denied**	Prevents accidental/malicious data loss.
Insert Row	Permitted	Permitted	Essential for daily operations.

Chapter 7

CONCLUSION

The integration of Python programming, SQL database management, and Power BI analytics forms a cohesive framework that addresses the complexities of the modern data landscape. This curriculum demonstrates that data proficiency is not merely about using a single tool, but about mastering the synergy between data retrieval, computational processing, and executive visualization.

SQL and Database Governance provide the essential bedrock for this ecosystem. As demonstrated through the experiments on Referential Integrity and User Management, a database is only as valuable as its reliability. By enforcing primary and foreign key constraints, we ensure "Entity Integrity," preventing the fragmentation of information. Furthermore, the implementation of Triggers and Views illustrates the shift toward automation and data abstraction—essential components for maintaining "Single Source of Truth" in high-volume environments.

Building upon this foundation, Python serves as the versatile engine for advanced manipulation. While SQL handles structured storage, Python's library ecosystem—specifically Pandas, NumPy, and Seaborn—allows for sophisticated exploratory data analysis and statistical modeling. The use of IPYWidgets and Streamlit further bridges the gap between static analysis and interactive application development, enabling analysts to create bespoke tools tailored to specific organizational needs.

The final transition into Power BI represents the conversion of raw data into "Actionable Intelligence." Through Data Modeling and Real-time Data Streaming, the curriculum highlights the importance of accessibility. An enterprise dashboard serves as the interface where complex SQL queries and Python scripts are distilled into intuitive visual narratives. The ability to publish these insights to a Power BI Server ensures that data-driven decision-making is democratized across the enterprise.

In conclusion, the convergence of these technologies ensures a robust data lifecycle: SQL manages the integrity, Python provides the analytical depth, and Power BI delivers the strategic clarity.